

CSE 398/498 Lab 2 Report:

Fine-Tuning Qwen3-VL and Evaluating Video QA on ReVA

Hengde Dai
hed424@lehigh.edu

September 20, 2026

GitHub repository (completed code, result files, report source):
<https://github.com/DylanDai63/cse498-lab2>

1 Overview

I completed the 18 `TODO(student)` functions in the four assignment files. The public unit tests went from 5 failed and 2 passed to 7 passed. I then ran the full workflow on the course MAGIC server: Qwen3-VL-4B-Instruct as the baseline, a LoRA fine-tuned Qwen3-VL-4B, and VILA1.5-3b as the second baseline. All three models answered the same 500 ReVA test questions. Table 1 gives the comparison that `scripts/compare_models.sh` writes to `outputs/model_comparison.csv`.

Model	Accuracy	Correct / questions	Answered
Qwen3-VL-4B base	65.2%	326 / 500	495
Qwen3-VL-4B fine-tuned (LoRA)	68.4%	342 / 500	500
VILA1.5-3b base	47.8%	239 / 500	500

Table 1: Results on the same 500-question subset of the ReVA test split. Accuracy divides by all 500 questions. An unanswered or unparsable prediction counts as incorrect.

Environment: `magic01.cse.lehigh.edu`, a shared server with four RTX 2080 Ti GPUs (11 GB each, Turing architecture). The `qwen2` conda environment uses Python 3.10, PyTorch 2.6.0 with CUDA 12.4, transformers 4.57.6, peft 0.17.1 and deepspeed 0.17.1. The `vila` environment uses PyTorch 2.3.0 and transformers 4.46.0.

2 Implementation of the TODO Functions

Part 1, `scripts/build_qwen_train_data.py`. `format_options` sorts the option labels because the annotation JSON does not guarantee A to D order. `format_question` starts with the `<video>` placeholder and, in the `reva_eval` style, copies the wording of the evaluation prompt in `reva_eval/eval_reva_v2.sh`, so the model sees the same instruction in training and in evaluation. `format_answer` upper-cases the letter and supports the three answer styles. `iter_reva_qas` flattens the four nested levels (video, category, question type, QA list) into one record per question. `convert_item` keeps the video path relative to `data/qwen_train`, which is the root that the Qwen data loader joins with the path. The function returns `None` for a missing video, so `main` can count skipped samples and refuse to write an empty training set.

Part 2, test-set preparation and scoring. The two files are `reva_eval/data/rsvidqa/prepare_reva_v2_test_set.py` and `scripts/score_reva_predictions.py`. `get_qa_id` prefers `qa_id`, then `global_index`, then an id built from the video file name, the question type and the question index. All three levels matter on the real data. The 4000 test questions contain 2582 ids of the first kind, 1393 of the second and 25 of the third, and I checked that all 4000 ids are unique. `extract_letter` parses in tiers from most to least explicit: a bare letter, a leading label such as “B.”, an “answer is C” cue, and finally a single stand-alone capital letter. Text inside `<think>` is removed first, and an ambiguous output returns an empty string instead of a guess. `score_predictions` loops over the ground truth, not over the predictions. A missing or unparsable prediction therefore stays in the denominator, and the scorer reports `Completed` separately from `Total`.

Part 3, vila_eval/reva_v2.py. `load_instances` uses the same id recipe as Part 2, so a Qwen prediction and a VILA prediction for one question share one id. `resolve_video_path` tries the raw path, the path under the dataset root, and the path with the dataset prefix removed, and raises an error when nothing resolves. `parse_choice` adds one tier to the Qwen parser. An answer given as option text is accepted when exactly one option matches. `summarize` reports `num_answered` next to accuracy, overall and per category.

3 Experimental Setup

Data. I downloaded ReVA from Hugging Face. The file `test_set.json` has the same MD5 checksum as the instructor copy in `/data/datasets/ReVA`. The test split has 4000 questions on 1014 videos, and every referenced video exists on disk.

Evaluation subset. A timing run on real data measured about 7.5 seconds per question per GPU for the Qwen baseline, because the prompt asks for step-by-step reasoning and the GPUs emulate bfloat16. One full evaluation would take about two hours on four GPUs of a server that other students share. I therefore evaluated all three models on one fixed subset of 500 questions. The script `scripts/make_eval_subset.py` draws the subset stratified by question type with seed 2026, pins the stable id of every question, and also writes the complementary 3500 questions. The remaining questions can be evaluated later and scored together with the subset, with no question evaluated twice. At 500 questions the standard error of one accuracy value is about ± 2.1 percentage points.

Training data and fine-tuning. I used the script default of 200 training samples. The 200 answers are balanced (50 for each letter), cover all 11 question types, and come from 17 videos. Only 8 of the 500 test questions use one of those 17 videos. Fine-tuning used the provided LoRA setup: rank 8, alpha 16, adapters on the q, k, v and o projections, 1 epoch, batch size 1 with 4 gradient accumulation steps (50 optimizer steps), and 4 frames per video. I changed one hyperparameter. The default learning rate of 2×10^{-7} suits full fine-tuning, but LoRA adapters start from zero and would barely move in 50 steps, so I used 1×10^{-4} . Training took 275 seconds on one GPU. The mean loss fell from 0.29 over the first ten steps to 0.11 over the last ten.

Evaluation settings. Both Qwen runs used the script defaults: `MAX_FRAMES=32`, 50176 pixels per frame, greedy decoding and at most 1024 new tokens. VILA used the script default of 4 frames and float16 weights.

Frames that reach Qwen. More than 99% of the `video_grid.thw` entries in the evaluation logs show a temporal grid of 2. A grid of 2 means 4 frames per video and not 32. The cause is in the provided inference script. The script samples 32 frames and passes the frames to the Qwen3-VL processor without video metadata. The processor then assumes 24 frames per second and samples

again. The formula $\lfloor 32/24 \times 2 \rfloor = 2$ falls below the minimum of 4, so the processor keeps frames 0, 10, 21 and 31. The processor also writes a text timestamp in front of each temporal patch, computed as the frame index divided by 24. Qwen therefore sees the labels `<0.2 seconds>` and `<1.1 seconds>` for a video of about 6 seconds. I kept the provided script unchanged so that the numbers stay comparable with other runs of the assignment. Qwen and VILA both saw 4 frames per video.

4 Hardware Adaptation

The provided scripts assume GPUs with FlashAttention 2 and native bfloat16 support. The RTX 2080 Ti has neither. I added the switches in Table 2. Every switch keeps the original behaviour when the variable is unset, so the code still runs unchanged on newer GPUs.

Problem on the server	Change	Setting I used
FlashAttention 2 does not support Turing GPUs	<code>trainer.py</code> imports <code>flash_attn</code> optionally; <code>train_qwen.py</code> reads the attention backend from a variable; <code>sft_7b.sh</code> reads <code>--data_flatten</code> from a variable	<code>ATTN_IMPLEMENTATION=sdpa</code> <code>DATA_FLATTEN=False</code>
<code>ValueError: Your setup doesn't support bf16/gpu</code>	<code>sft_7b.sh</code> selects the precision flag; <code>train_qwen.py</code> can load the frozen base weights in bfloat16 without the flag. PEFT keeps the LoRA weights in float32	<code>PRECISION=none</code> <code>MODEL_DTYPE=bfloat16</code>
<code>RuntimeError: Expected to mark a variable ready only once</code> (LoRA with reentrant gradient checkpointing under DDP)	<code>sft_7b.sh</code> appends extra Trainer flags	<code>EXTRA_TRAIN_ARGS</code> with the two flags shown in Section 8
CUDA out of memory in evaluation with the hard-coded batch of 8	<code>inference_vllm_origin_number.py</code> reads the batch size from a variable	<code>EVAL_BATCH_SIZE=4</code>
GPU 0 is often busy on the shared server	<code>eval_reva_v2.sh</code> accepts a list of physical GPU ids	<code>GPU_IDS="2 3"</code>
<code>torchcodec</code> cannot load without FFmpeg	Installed FFmpeg 7 from conda-forge into the environment	<code>none</code>

Table 2: Hardware adaptations. All defaults reproduce the original scripts.

VILA needed three local patches to the NVlabs/VILA repository, which I saved as `vila_eval/vila_local_patches.diff`. The SigLIP vision tower now loads with SDPA attention instead of FlashAttention 2. The PS3 vision tower import is optional, because `ps3-torch` conflicts with the `timm` version that VILA needs. The video loader treats a null `fps` value in the VILA1.5 configuration as zero. I also installed triton 3.1.0, as the official VILA setup script does.

5 Results

Table 3 lists accuracy by question type. The three time-related types (Temporal Grounding, Trend and Pattern, Change Detection) are the weakest types for both Qwen models, and Temporal Grounding is the weakest type for VILA. VILA scores 27.5% on Temporal Grounding, which is

close to the 25% chance level of a four-option question. The frame and timestamp issue from Section 3 is a likely reason for the Qwen results on this type. Two temporal positions with the labels 0.2 and 1.1 seconds cannot separate options that differ by 0.5 seconds in a 6-second video. The baseline outputs agree with this explanation. In 13 of the 80 Temporal Grounding questions, every time that the baseline states in its reasoning is at most 1.4 seconds, and the baseline answers 2 of these 13 questions correctly.

Question type	n	Qwen base	Qwen fine-tuned	VILA	Gained / lost
Causation Reasoning	23	95.7	91.3	78.3	+0 / -1
Change Detection	63	61.9	63.5	47.6	+7 / -6
Consequence Reasoning	13	84.6	92.3	92.3	+1 / -0
General Understanding	22	95.5	100.0	90.9	+1 / -0
Geometric Relation	50	78.0	74.0	48.0	+0 / -2
Hypothetical Reasoning	20	85.0	85.0	55.0	+1 / -1
Object and Land Cover Recognition	82	72.0	72.0	43.9	+3 / -3
Perspective and Viewpoint	60	65.0	75.0	36.7	+11 / -5
Structural Layout	42	69.0	64.3	59.5	+3 / -5
Temporal Grounding	80	33.8	45.0	27.5	+16 / -7
Trend and Pattern	45	51.1	57.8	42.2	+3 / -0

Table 3: Accuracy (%) by question type. The last column counts questions that changed between the Qwen baseline and the fine-tuned model.

Effect of fine-tuning. Accuracy rose from 65.2% to 68.4%. Both Qwen runs answered the same questions, so I compared the two runs question by question. Fine-tuning fixed 46 questions and broke 30, a net gain of 16. An exact McNemar test on the 46 and 30 changed questions gives $p = 0.085$. The improvement is suggestive but not statistically significant at 500 questions. Most of the gain comes from Temporal Grounding and from Perspective and Viewpoint. Change Detection and Object Recognition mostly trade one question for another.

Output format. Fine-tuning changed the output style more than it changed accuracy. The baseline wrote a median of 297 characters, and only 141 of 500 outputs started directly with the `<answer>` tag. After fine-tuning all 500 outputs are exactly `<answer>X</answer>` (18 characters). The training targets contain only the tagged letter, and the model adopted the target format. The five unparsable baseline outputs disappeared as a result.

Letter preference. The ground truth is balanced across A, B, C and D (128, 119, 123, 130). The Qwen baseline chose C 146 times and the fine-tuned model chose C 165 times. The 200 training answers are balanced, so the preference does not come from the training labels. VILA prefers A (174 times) and rarely chooses C (67 times). Each model appears to carry a letter prior of its own.

6 Qualitative Analysis

The five cases follow the order of two correct answers, two incorrect answers and one error caused by the output format. “Correct” and “incorrect” refer to the fine-tuned model, and each case lists the answers of all three models. Across the 500 questions, all three models are correct on 175 questions and all three are wrong on 93. Fine-tuning fixed 46 questions and broke 30.

Case 1, correct, visual perception (QA-001119, Object and Land Cover Recognition). The question asks for the most prominent land cover on the left side of the lower road. The annotation marks C, a dense row of yellow-green trees. The baseline, the fine-tuned model and VILA all answered C. The baseline described a long, continuous row of yellow-green trees and

ruled out bare soil, crops and industrial buildings. The target is large, stays in view for the whole video and does not change over time. Four frames carry enough evidence for such a question, and the answer does not depend on timing, which is the information that the evaluation pipeline loses.

Case 2, correct after fine-tuning, temporal reasoning (QA-001348, Temporal Grounding). The question asks for the time span in which a blue car stays in the frame. The annotation marks C, from 0.0 to 6.0 seconds, which is the full video. The baseline answered D, the fine-tuned model answered C, and VILA answered A. The baseline wrote that the car is no longer visible by 0.4 seconds, noted that no option has a duration of 0.4 seconds, and chose D as the closest option. The value 0.4 seconds lies on the compressed time axis from Section 3, where the last temporal patch carries the label 1.1 seconds. The baseline reasoning fails at the step that maps the stated time to the options. The fine-tuned model gave the correct letter without any reasoning. The run therefore cannot show whether the fine-tuned model localizes time better or only stopped reasoning on the misleading labels.

Case 3, incorrect, temporal reasoning (QA-001983, Temporal Grounding). The question asks how long a white bus remains visible. All four options last 2.0 seconds and differ only by a 0.5-second shift of the start time. The annotation marks D, from 2.5 to 4.5 seconds. The baseline described the scene in 0.5-second steps, reported the bus as still visible at 6.0 seconds, and reached the token limit without an answer. The fine-tuned model answered C and VILA answered B. All three models failed. With four frames per video, no model receives enough temporal resolution to separate start times that differ by 0.5 seconds. Fine-tuning changed the output format of Qwen but could not add the missing temporal information.

Case 4, incorrect after fine-tuning, overfitting to the answer format (QA-008909, Causation Reasoning). The question asks for the most likely cause of a severe traffic jam. The annotation marks D, vehicles queuing to refuel at the adjacent gas station. The baseline and VILA answered D, and the fine-tuned model answered C, a construction project. The baseline wrote that the video shows no sign of construction, accidents or a major event, and pointed to the gas station beside the road. Construction is the cause that the baseline had ruled out from the video. The fine-tuned model gave no reasoning. Of the 30 questions that fine-tuning broke, 15 received the answer C. The pattern suggests that the model learned to skip the reasoning step and, without reasoning, falls back on a letter prior. The available runs do not test the explanation directly.

Case 5, output format error, option parsing (REVA-G001758, Temporal Grounding). The question asks when the first vehicle appears on the bridge. The options are 0.0, 5.0, 1.5 and 3.0 seconds, and the annotation marks A, 0.0 seconds. The baseline wrote that the bridge is empty at 0.0 seconds and that the vehicle appears at 0.5 seconds, again a value on the compressed time axis. Because 0.5 seconds is not an option, the baseline alternated between A and C, repeated “But wait”, and reached the 1024-token limit without an `<answer>` tag. My scorer returns no letter for such an output, because choosing A or C from the text would be a guess, and the question counts as incorrect. The fine-tuned model answered `<answer>A</answer>`, which is correct, and VILA answered D. Four of the five unparsable baseline outputs are Temporal Grounding questions that end at the token limit in the same way. The fifth (QA-009967) is a visual perception failure. The baseline did not see the small body of water in the video and answered “None of the above”, which is not among the options.

7 Limitations

The numbers come from 500 of the 4000 test questions and from one training run with one seed. The first 277 baseline questions ran with an evaluation batch of 8 before the out-of-memory crash,

and the rest ran with a batch of 4. Decoding is greedy, so the batch size can change an answer only through padding. The frame and timestamp issue from Section 3 affects every Qwen number in the report, and the Temporal Grounding numbers most of all.

8 Commands

The repository README lists every command in order. The three evaluation runs and the training run were:

```
python scripts/make_eval_subset.py --input $REVA/test_set.json --size 500 --seed 2026

NUM_CHUNKS=4 EVAL_NAME=qwen_base REVA_ROOT=$REVA REVA_JSON=$SUBSET MODEL_PATH=$QWEN \
  bash scripts/run_eval_qwen_base.sh      # default batch 8: CUDA OOM after 277 questions
RESUME=1 EVAL_BATCH_SIZE=4 NUM_CHUNKS=4 EVAL_NAME=qwen_base REVA_ROOT=$REVA \
  REVA_JSON=$SUBSET MODEL_PATH=$QWEN bash scripts/run_eval_qwen_base.sh      # resumed

CUDA_VISIBLE_DEVICES=3 ATTN_IMPLEMENTATION=sdpa DATA_FLATTEN=False PRECISION=none \
  MODEL_DTYPE=bfloat16 USE_DEEPSPEED=0 NPROC_PER_NODE=1 LR=1e-4 MODEL_PATH=$QWEN \
  EXTRA_TRAIN_ARGS='--ddp_find_unused_parameters False \
    --gradient_checkpointing_kwargs {"use_reentrant":false}' \
  bash scripts/run_finetune_qwen.sh

GPU_IDS="2 3" NUM_CHUNKS=2 EVAL_BATCH_SIZE=4 EVAL_NAME=qwen_sft REVA_ROOT=$REVA \
  REVA_JSON=$SUBSET MODEL_BASE=$QWEN MODEL_PATH=outputs/qwen_reva_sft/checkpoint-50 \
  bash scripts/run_eval_qwen_finetuned.sh

GPU=1 VILA_REPO=$VILA MODEL_PATH=$VILA_MODEL REVA_ROOT=$REVA REVA_JSON=$SUBSET \
  bash scripts/run_eval_reva_vila.sh

bash scripts/compare_models.sh
```

AI Use Statement

I used Claude (Anthropic) as an assistant throughout the lab: for setting up the server environments, for diagnosing the hardware incompatibilities in Section 4, for implementing and testing the TODO functions, for the analysis scripts, and for drafting the report text. I ran every command on the server myself and reviewed the code and the results.